PROJECT ON SQL

Hospital Patient Management System

SUBJECT: Project on SQL

TOPIC: Hospital Patient Management System

DATABASE: MediTrack

CONTENTS

1) Project Description

2) Entity Relationship Diagram

3) Code

4) Questions and Answers

5) Source Files

6) References

PROJECT DESCRIPTION

Overview

DBMS (Database Management System):

A DBMS is software that lets users create, manage, and interact with databases. It provides tools for storing, retrieving, updating, and securing data, while ensuring data integrity, reducing redundancy, and supporting multiple concurrent users. Examples include MySQL, PostgreSQL, and Oracle.

RDBMS (Relational Database Management System):

An RDBMS organizes data into structured tables made up of rows (records) and columns (attributes), and links related tables through relationships. RDBMS platforms rely on SQL for querying and managing data. Popular examples include MySQL, SQL Server, and SQLite.

SQL (Structured Query Language):

SQL is the standard language for interacting with relational databases. It supports querying data (SELECT), inserting records (INSERT), updating data (UPDATE), and deleting records (DELETE), as well as defining database structures (CREATE, ALTER, DROP) and managing user permissions (GRANT, REVOKE).

Normalization

Normalization organizes data to reduce redundancy and improve integrity by breaking large tables into smaller, related tables. The main normal forms are:

- 1NF (First Normal Form): Every column holds only atomic (indivisible) values.
- 2NF (Second Normal Form): Removes partial dependencies so non-key attributes depend on the whole primary key.
- 3NF (Third Normal Form): Removes transitive dependencies so non-key attributes depend only on the primary key.
- BCNF (Boyce-Codd Normal Form): A stricter version of 3NF where every determinant is a candidate key.

Keys

Keys are attributes, or sets of attributes, that uniquely identify a row within a table.

- Primary Key: Uniquely identifies each record in a table; cannot be null.
- Foreign Key: A field in one table that references the primary key of another, establishing a relationship.
- Composite Key: Two or more columns combined to form a unique identifier.
- Candidate Key: A column, or set of columns, that could serve as the primary key but wasn't chosen.
- Unique Key: Guarantees unique values in a column but, unlike a primary key, may contain a null.

Relationships

- One-to-One (1:1): Each record in one table relates to exactly one record in another, e.g. an appointment and its bill.
- One-to-Many (1:M): A record in one table relates to many records in another, e.g. a patient and their appointments.
- Many-to-Many (M:M): Many records in one table relate to many in another, usually resolved with a junction table, e.g. doctors and patients across many appointments.

Indexes

An index is a database object that speeds up data retrieval by letting the engine locate rows without scanning the whole table — much like an index at the back of a book.

- Single-Column Index: Built on one column.
- Composite Index: Built across multiple columns.
- Unique Index: Guarantees every value in the indexed column(s) is distinct.

Transactions

A transaction is a sequence of operations executed as a single logical unit of work, keeping the database consistent even with many simultaneous users.

- START TRANSACTION: Begins a new transaction.
- COMMIT: Permanently saves all changes made during the transaction.
- ROLLBACK: Undoes all changes made during the transaction if something goes wrong.

ACID Properties

- Atomicity: Every operation in a transaction succeeds, or none of them do.
- Consistency: The database moves from one valid state to another.
- Isolation: Concurrent transactions do not interfere with one another.
- Durability: Once committed, changes survive even a system failure.

Views

A view is a virtual table produced by a stored query. It doesn't hold data itself but presents data drawn from one or more underlying tables — useful for simplifying complex queries, restricting access to specific columns or rows, and giving users a stable interface even if the underlying tables change. Views are created with CREATE VIEW and queried like ordinary tables.

MediTrack — Hospital Patient Management Database

MediTrack is a hospital patient-management system backed by a relational database built in MySQL. It tracks patients, doctors, appointments, prescriptions, and billing so that front-desk staff, physicians, and accounts teams can all work from one consistent source of data. The system is organized around five core tables: Patients, Doctors, Appointments, Prescriptions, and Billing. Each table stores one kind of essential information, and relationships between the tables support efficient scheduling, clinical record-keeping, and financial reporting.

The database is normalized to avoid duplicating information — patient details are stored once and referenced from every appointment, doctor details are stored once and referenced from every consultation, and so on. This keeps

updates simple (e.g. correcting a patient's contact number in a single place) and keeps reporting accurate as the hospital's patient volume grows.

Overall, the MediTrack database is a compact but realistic example of how a relational schema supports day-to-day operations — scheduling appointments, prescribing medicine, generating bills, and producing management reports — while keeping the underlying data consistent and easy to maintain.

ENTITY RELATIONSHIP DIAGRAM

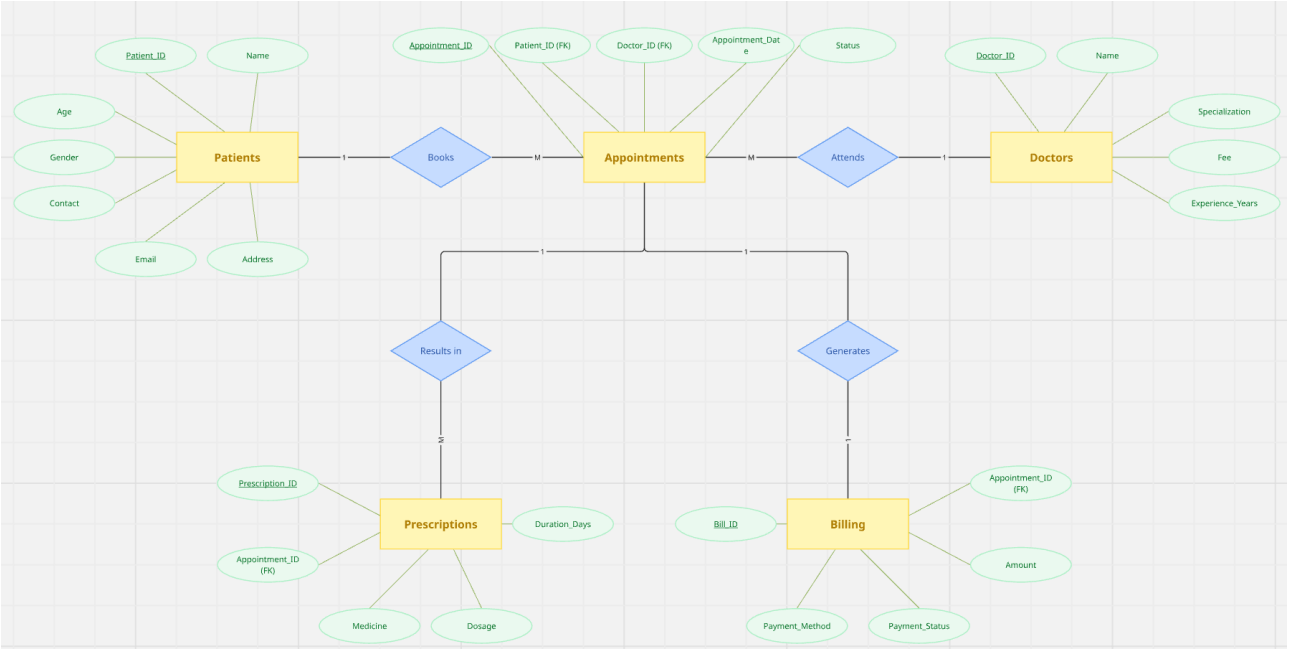
An ERD is a blueprint for database design. It defines the entities in a system, the attributes that describe each entity, and the relationships that connect them — making it an essential planning tool before any table is created.

Entities and Attributes

Entity	Attributes
Patients	Patient_ID (PK), Name, Age, Gender, Contact, Email, Address
Doctors	Doctor_ID (PK), Name, Specialization, Fee, Experience_Years
Appointments	Appointment_ID (PK), Patient_ID (FK), Doctor_ID (FK), Appointment_Date, Status
Prescriptions	Prescription_ID (PK), Appointment_ID (FK), Medicine, Dosage, Duration_Days
Billing	Bill_ID (PK), Appointment_ID (FK), Amount, Payment_Method, Payment_Status

Key Relations

- Patients and Appointments (1:M) — A single patient can book multiple appointments, but each appointment belongs to only one patient.
- Doctors and Appointments (1:M) — A single doctor can attend multiple appointments, but each appointment is attended by only one doctor.
- Appointments and Prescriptions (1:M) — A single appointment can result in multiple prescribed medicines, but each prescription line belongs to only one appointment.
- Appointments and Billing (1:1) — Each appointment generates exactly one bill, and each bill corresponds to exactly one appointment.

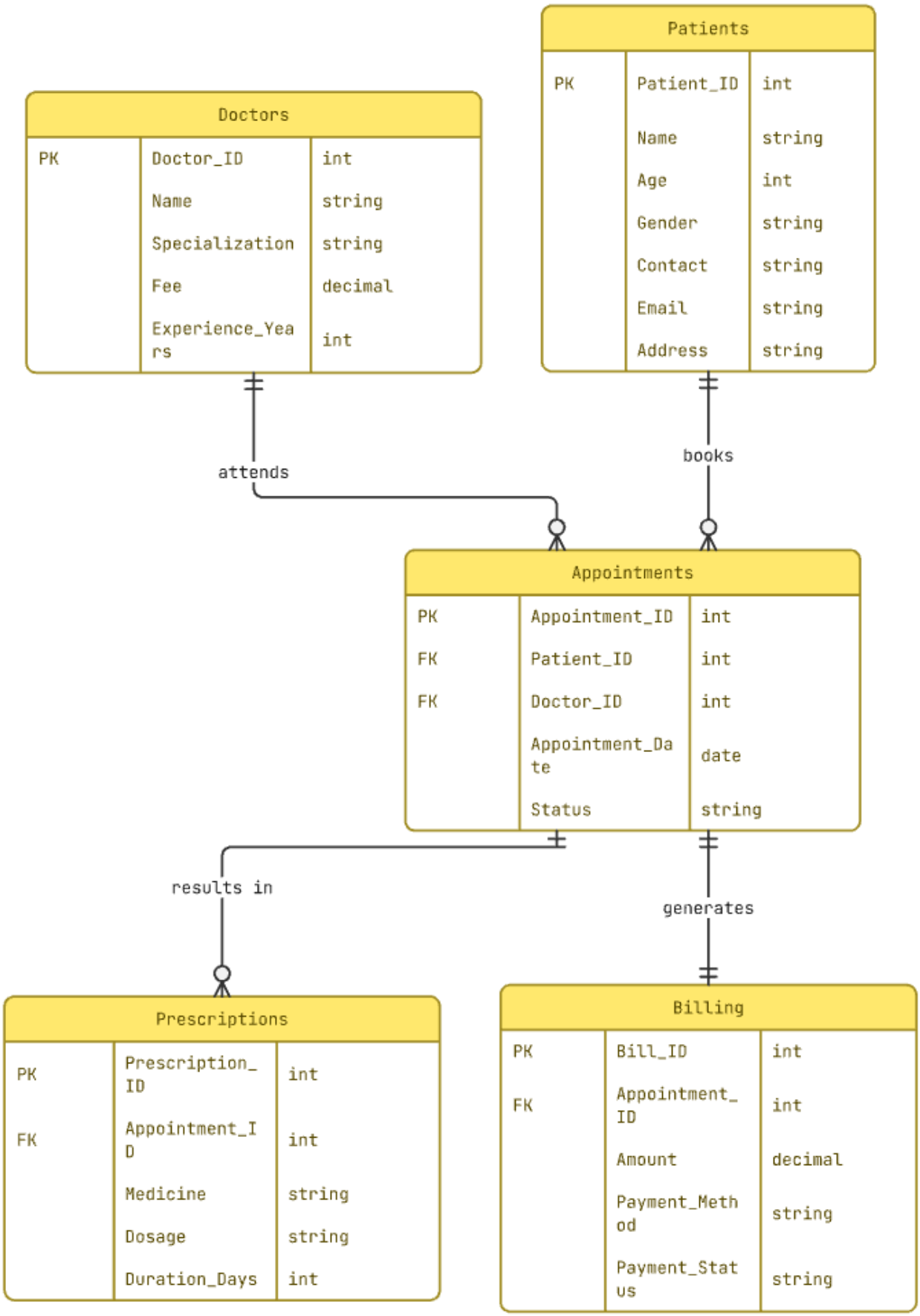


ENTITY RELATIONSHIP DIAGRAM

Notes

- Implemented using Patient_ID as a foreign key in the Appointments table, referencing Patient_ID in Patients.

- Implemented using Doctor_ID as a foreign key in the Appointments table, referencing Doctor_ID in Doctors.
- Implemented using Appointment_ID as a foreign key in the Prescriptions table, referencing Appointment_ID in Appointments.
- Implemented using Appointment_ID as a foreign key in the Billing table, referencing Appointment_ID in Appointments.
- Prescription_ID is a surrogate primary key in Prescriptions, since a single appointment can legitimately generate several prescription lines for the same or different medicines.


Patients
PK
Patient_ID
int
Name
string
Age
int
Gender
string
Contact
string
Email
string
Address
string
Doctors
PK
Doctor_ID
int
Name
string
Specialization
string
Fee
decimal
Experience_Years
int
attends
books
Appointments
PK
Appointment_ID
int
FK
Patient_ID
int
FK
Doctor_ID
int
Appointment_Date
date
Status
string
results in
generates
Prescriptions
PK
Prescription_ID
int
FK
Appointment_ID
int
Medicine
string
Dosage
string
Duration_Days
int
Billing
PK
Bill_ID
int
FK
Appointment_ID
int
Amount
decimal
Payment_Method
string
Payment_Status
string

CODE

1. Create Database

```
mysql> CREATE DATABASE MediTrack;
Query OK, 1 row affected (0.06 sec)

mysql> USE MediTrack;
Database changed
```

2. Patients Table

```
mysql> CREATE TABLE Patients (
    Patient_ID INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT,
    Gender VARCHAR(10),
    Contact VARCHAR(15),
    Email VARCHAR(100),
    Address VARCHAR(100)
  );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Patients VALUES
(1, "Rohit Sharma", 34, "M", "9831000001", "rohit.sharma@gmail.com", "Kolkata"), (2, "Simran
Kaur", 29, "F", "9831000002", "simran.kaur@yahoo.com", "Delhi"),
(3, "Amit Ghosh", 45, "M", "9831000003", "amit.ghosh@gmail.com", "Mumbai"), (4, "Nisha Patel", 51,
"F", "9831000004", "nisha.patel@hotmail.com", "Ahmedabad"),
(5, "Suresh Reddy", 60, "M", "9831000005", "suresh.reddy@gmail.com", "Hyderabad"), (6, "Meera
Nair", 38, "F", "9831000006", "meera.nair@yahoo.com", "Chennai"),
(7, "Vikas Yadav", 27, "M", "9831000007", "vikas.yadav@outlook.com", "Lucknow"), (8, "Anjali
Gupta", 33, "F", "9831000008", "anjali.gupta@gmail.com", "Jaipur"),
(9, "Rajesh Kumar", 55, "M", "9831000009", "rajesh.kumar@yahoo.com", "Pune"), (10, "Pooja Singh",
41, "F", "9831000010", "pooja.singh@gmail.com", "Bhopal"),
(11, "Sanjay Verma", 48, "M", "9831000011", "sanjay.verma@outlook.com", "Kolkata"), (12, "Kiran
Das", 36, "F", "9831000012", "kiran.das@gmail.com", "Delhi"),
(13, "Manoj Tiwari", 29, "M", "9831000013", "manoj.tiwari@yahoo.com", "Mumbai"), (14, "Ritu
Chatterjee", 44, "F", "9831000014", "ritu.chatterjee@gmail.com", "Kolkata"),
(15, "Deepak Joshi", 52, "M", "9831000015", "deepak.joshi@outlook.com", "Ahmedabad"), (16, "Swati
Bose", 31, "F", "9831000016", "swati.bose@gmail.com", "Hyderabad"),
(17, "Arvind Rao", 39, "M", "9831000017", "arvind.rao@yahoo.com", "Chennai"), (18, "Nandini Sen",
26, "F", "9831000018", "nandini.sen@gmail.com", "Lucknow"),
(19, "Harish Chandra", 58, "M", "9831000019", "harish.chandra@outlook.com", "Jaipur"), (20, "Lata
Mishra", 47, "F", "9831000020", "lata.mishra@gmail.com", "Pune");
```

```
Query OK, 20 rows affected (0.01 sec)
Records: 20  Duplicates: 0  Warnings: 0
```

mysql> SELECT * FROM Patients;

Patient_ID	Name	Age	Gender	Contact	Email	Address
1	Rohit Sharma	34	M	9831000001	rohit.sharma@gmail.com	Kolkata
2	Simran Kaur	29	F	9831000002	simran.kaur@yahoo.com	Delhi

Patient_ID	Name	Age	Gender	Contact	Email	Address
3	Amit Ghosh	45	M	9831000003	amit.ghosh@gmail.com	Mumbai
4	Nisha Patel	51	F	9831000004	nisha.patel@hotmail.com	Ahmedabad
5	Suresh Reddy	60	M	9831000005	suresh.reddy@gmail.com	Hyderabad
6	Meera Nair	38	F	9831000006	meera.nair@yahoo.com	Chennai
7	Vikas Yadav	27	M	9831000007	vikas.yadav@outlook.com	Lucknow
8	Anjali Gupta	33	F	9831000008	anjali.gupta@gmail.com	Jaipur
9	Rajesh Kumar	55	M	9831000009	rajesh.kumar@yahoo.com	Pune
10	Pooja Singh	41	F	9831000010	pooja.singh@gmail.com	Bhopal
11	Sanjay Verma	48	M	9831000011	sanjay.verma@outlook.com	Kolkata
12	Kiran Das	36	F	9831000012	kiran.das@gmail.com	Delhi
13	Manoj Tiwari	29	M	9831000013	manoj.tiwari@yahoo.com	Mumbai
14	Ritu Chatterjee	44	F	9831000014	ritu.chatterjee@gmail.com	Kolkata
15	Deepak Joshi	52	M	9831000015	deepak.joshi@outlook.com	Ahmedabad
16	Swati Bose	31	F	9831000016	swati.bose@gmail.com	Hyderabad
17	Arvind Rao	39	M	9831000017	arvind.rao@yahoo.com	Chennai
18	Nandini Sen	26	F	9831000018	nandini.sen@gmail.com	Lucknow
19	Harish Chandra	58	M	9831000019	harish.chandra@outlook.com	Jaipur
20	Lata Mishra	47	F	9831000020	lata.mishra@gmail.com	Pune

20 rows in set

3. Doctors Table

```
mysql> CREATE TABLE Doctors (
    Doctor_ID INT PRIMARY KEY,
    Name VARCHAR(100),
    Specialization VARCHAR(50),
    Fee DECIMAL(10, 2),
    Experience_Years INT
  );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO Doctors VALUES
(201, "Dr. Rohan Mehta", "Cardiology", "800.00", 12), (202, "Dr. Ananya Roy", "Dermatology",
"500.00", 8),
(203, "Dr. Vikram Singh", "Orthopedics", "700.00", 15), (204, "Dr. Priya Nair", "Pediatrics",
"450.00", 10),
(205, "Dr. Arjun Kapoor", "Neurology", "900.00", 18), (206, "Dr. Sneha Iyer", "Gynecology",
"600.00", 9),
(207, "Dr. Karan Malhotra", "ENT", "400.00", 7), (208, "Dr. Divya Menon", "Ophthalmology",
"550.00", 11),
(209, "Dr. Rahul Verma", "General Medicine", "300.00", 6), (210, "Dr. Neha Joshi", "Psychiatry",
"650.00", 13),
(211, "Dr. Sameer Khan", "Urology", "750.00", 14), (212, "Dr. Pooja Desai", "Endocrinology",
"700.00", 10),
(213, "Dr. Aditya Rao", "Gastroenterology", "800.00", 16), (214, "Dr. Kavita Pillai", "Oncology",
"1000.00", 20),
(215, "Dr. Manish Agarwal", "Nephrology", "850.00", 17);
```

```
Query OK, 15 rows affected (0.01 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

mysql> SELECT * FROM Doctors;

Doctor_ID	Name	Specialization	Fee	Experience_Years
201	Dr. Rohan Mehta	Cardiology	800.00	12
202	Dr. Ananya Roy	Dermatology	500.00	8
203	Dr. Vikram Singh	Orthopedics	700.00	15
204	Dr. Priya Nair	Pediatrics	450.00	10
205	Dr. Arjun Kapoor	Neurology	900.00	18
206	Dr. Sneha Iyer	Gynecology	600.00	9
207	Dr. Karan Malhotra	ENT	400.00	7
208	Dr. Divya Menon	Ophthalmology	550.00	11
209	Dr. Rahul Verma	General Medicine	300.00	6
210	Dr. Neha Joshi	Psychiatry	650.00	13
211	Dr. Sameer Khan	Urology	750.00	14
212	Dr. Pooja Desai	Endocrinology	700.00	10
213	Dr. Aditya Rao	Gastroenterology	800.00	16
214	Dr. Kavita Pillai	Oncology	1000.00	20
215	Dr. Manish Agarwal	Nephrology	850.00	17

15 rows in set

4. Appointments Table

```
mysql> CREATE TABLE Appointments (
    Appointment_ID INT PRIMARY KEY,
    Patient_ID INT,
    Doctor_ID INT,
    Appointment_Date DATE,
    Status VARCHAR(20),
    FOREIGN KEY (Patient_ID) REFERENCES Patients(Patient_ID),
    FOREIGN KEY (Doctor_ID) REFERENCES Doctors(Doctor_ID)
  );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO Appointments VALUES
(3001, 1, 201, "2024-01-05", "Completed"), (3002, 3, 205, "2024-01-06", "Completed"),
(3003, 5, 203, "2024-01-07", "Completed"), (3004, 7, 209, "2024-01-08", "Completed"),
(3005, 10, 214, "2024-01-09", "Completed"), (3006, 2, 206, "2024-01-10", "Completed"),
(3007, 4, 202, "2024-01-11", "Completed"), (3008, 6, 208, "2024-01-12", "Completed"),
(3009, 8, 213, "2024-01-13", "Completed"), (3010, 9, 210, "2024-01-14", "Completed");
```

```
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

mysql> SELECT * FROM Appointments;

Appointment_ID	Patient_ID	Doctor_ID	Appointment_Date	Status
3001	1	201	2024-01-05	Completed
3002	3	205	2024-01-06	Completed
3003	5	203	2024-01-07	Completed
3004	7	209	2024-01-08	Completed
3005	10	214	2024-01-09	Completed
3006	2	206	2024-01-10	Completed
3007	4	202	2024-01-11	Completed
3008	6	208	2024-01-12	Completed
3009	8	213	2024-01-13	Completed
3010	9	210	2024-01-14	Completed

10 rows in set

5. Prescriptions Table

```
mysql> CREATE TABLE Prescriptions (
    Prescription_ID INT PRIMARY KEY,
    Appointment_ID INT,
    Medicine VARCHAR(100),
    Dosage VARCHAR(20),
    Duration_Days INT,
    FOREIGN KEY (Appointment_ID) REFERENCES Appointments(Appointment_ID)
  );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO Prescriptions VALUES
(5001, 3001, "Aspirin", "75mg", 30), (5002, 3001, "Atorvastatin", "20mg", 30),
(5003, 3002, "Levetiracetam", "500mg", 60), (5004, 3002, "Gabapentin", "300mg", 30),
(5005, 3003, "Ibuprofen", "400mg", 10), (5006, 3003, "Calcium", "500mg", 30),
(5007, 3004, "Paracetamol", "500mg", 5), (5008, 3005, "Tamoxifen", "20mg", 90),
(5009, 3006, "Folic Acid", "5mg", 30), (5010, 3007, "Cetirizine", "10mg", 15),
(5011, 3008, "Timolol Eye Drops", "0.5%", 30), (5012, 3009, "Omeprazole", "20mg", 14),
(5013, 3010, "Sertraline", "50mg", 60);
```

```
Query OK, 13 rows affected (0.01 sec)
Records: 13  Duplicates: 0  Warnings: 0
```

mysql> SELECT * FROM Prescriptions;

Prescription_ID	Appointment_ID	Medicine	Dosage	Duration_Days
5001	3001	Aspirin	75mg	30
5002	3001	Atorvastatin	20mg	30
5003	3002	Levetiracetam	500mg	60
5004	3002	Gabapentin	300mg	30
5005	3003	Ibuprofen	400mg	10
5006	3003	Calcium	500mg	30
5007	3004	Paracetamol	500mg	5
5008	3005	Tamoxifen	20mg	90
5009	3006	Folic Acid	5mg	30
5010	3007	Cetirizine	10mg	15
5011	3008	Timolol Eye Drops	0.5%	30
5012	3009	Omeprazole	20mg	14
5013	3010	Sertraline	50mg	60

13 rows in set

6. Billing Table

```
mysql> CREATE TABLE Billing (
    Bill_ID INT PRIMARY KEY,
    Appointment_ID INT,
    Amount DECIMAL(10, 2),
    Payment_Method VARCHAR(50),
    Payment_Status VARCHAR(20),
    FOREIGN KEY (Appointment_ID) REFERENCES Appointments(Appointment_ID)
  );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO Billing VALUES
(4001, 3001, "800.00", "Credit Card", "Paid"), (4002, 3002, "900.00", "Credit Card", "Paid"),
(4003, 3003, "700.00", "Cash", "Paid"), (4004, 3004, "300.00", "Debit Card", "Paid"),
(4005, 3005, "1000.00", "Credit Card", "Paid"), (4006, 3006, "600.00", "UPI", "Paid"),
(4007, 3007, "500.00", "Cash", "Paid"), (4008, 3008, "550.00", "Debit Card", "Paid"),
(4009, 3009, "800.00", "Credit Card", "Paid"), (4010, 3010, "650.00", "UPI", "Paid");
```

```
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

mysql> SELECT * FROM Billing;

Bill_ID	Appointment_ID	Amount	Payment_Method	Payment_Status
4001	3001	800.00	Credit Card	Paid
4002	3002	900.00	Credit Card	Paid
4003	3003	700.00	Cash	Paid
4004	3004	300.00	Debit Card	Paid
4005	3005	1000.00	Credit Card	Paid
4006	3006	600.00	UPI	Paid
4007	3007	500.00	Cash	Paid
4008	3008	550.00	Debit Card	Paid
4009	3009	800.00	Credit Card	Paid
4010	3010	650.00	UPI	Paid

10 rows in set

QUESTIONS AND ANSWERS

Q1. List all appointments scheduled after '2024-01-08'.

Query:

```
SELECT * FROM Appointments WHERE Appointment_Date > '2024-01-08';
```

Output:

Appointment_ID	Patient_ID	Doctor_ID	Appointment_Date	Status
3005	10	214	2024-01-09	Completed
3006	2	206	2024-01-10	Completed
3007	4	202	2024-01-11	Completed
3008	6	208	2024-01-12	Completed
3009	8	213	2024-01-13	Completed
3010	9	210	2024-01-14	Completed

6 rows in set

Q2. Retrieve all payments made via 'Credit Card'.

Query:

```
SELECT * FROM Billing WHERE Payment_Method = 'Credit Card';
```

Output:

Bill_ID	Appointment_ID	Amount	Payment_Method	Payment_Status
4001	3001	800.00	Credit Card	Paid
4002	3002	900.00	Credit Card	Paid
4005	3005	1000.00	Credit Card	Paid
4009	3009	800.00	Credit Card	Paid

4 rows in set

Q3. Display the 5 doctors with the highest consultation fees.

Query:

```
SELECT * FROM Doctors ORDER BY Fee DESC LIMIT 5;
```

Output:

Doctor_ID	Name	Specialization	Fee	Experience_Years
214	Dr. Kavita Pillai	Oncology	1000.00	20
205	Dr. Arjun Kapoor	Neurology	900.00	18
215	Dr. Manish Agarwal	Nephrology	850.00	17
201	Dr. Rohan Mehta	Cardiology	800.00	12
213	Dr. Aditya Rao	Gastroenterology	800.00	16

5 rows in set

Q4. Show the 3 most recent appointments.

Query:

```
SELECT * FROM Appointments ORDER BY Appointment_Date DESC LIMIT 3;
```

Output:

Appointment_ID	Patient_ID	Doctor_ID	Appointment_Date	Status
3010	9	210	2024-01-14	Completed
3009	8	213	2024-01-13	Completed
3008	6	208	2024-01-12	Completed

3 rows in set

Q5. Find all medicines prescribed to 'Rohit Sharma'.

Query:

```
SELECT pr.Medicine FROM Prescriptions pr
  JOIN Appointments a ON pr.Appointment_ID = a.Appointment_ID
  JOIN Patients p ON a.Patient_ID = p.Patient_ID
  WHERE p.Name = 'Rohit Sharma';
```

Output:

Medicine
Aspirin
Atorvastatin

2 rows in set

Q6. Display bills with an amount greater than the average billing amount.

Query:

```
SELECT * FROM Billing WHERE Amount > (SELECT AVG(Amount) FROM Billing);
```

Output:

Bill_ID	Appointment_ID	Amount	Payment_Method	Payment_Status
4001	3001	800.00	Credit Card	Paid
4002	3002	900.00	Credit Card	Paid
4003	3003	700.00	Cash	Paid
4005	3005	1000.00	Credit Card	Paid
4009	3009	800.00	Credit Card	Paid

5 rows in set

Average billing amount = 680.00

Q7. Find the total number of appointments booked by each patient.

Query:

```
SELECT Patient_ID, COUNT(*) AS Total_Appointments FROM Appointments GROUP BY Patient_ID;
```

Output:

Patient_ID	Total_Appointments
1	1
3	1
5	1
7	1
10	1
2	1
4	1
6	1
8	1
9	1

10 rows in set

Q8. Find the average consultation fee for each specialization.

Query:

```
SELECT Specialization, AVG(Fee) AS Avg_Fee FROM Doctors GROUP BY Specialization;
```

Output:

Specialization	Avg_Fee
Cardiology	800.00
Dermatology	500.00
Orthopedics	700.00
Pediatrics	450.00
Neurology	900.00
Gynecology	600.00
ENT	400.00
Ophthalmology	550.00
General Medicine	300.00
Psychiatry	650.00
Urology	750.00
Endocrinology	700.00
Gastroenterology	800.00
Oncology	1000.00
Nephrology	850.00

15 rows in set

Q9. Show the number of completed appointments.

Query:

```
SELECT COUNT(*) AS Completed_Appointments FROM Appointments WHERE Status = 'Completed';
```

Output:

Completed_Appointments
10

1 row in set

Q10. Find the patient who has spent the most on medical bills.

Query:

```
SELECT a.Patient_ID, SUM(b.Amount) AS Total_Spent
  FROM Appointments a JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY a.Patient_ID ORDER BY Total_Spent DESC LIMIT 1;
```

Output:

Patient_ID	Total_Spent
10	1000.00

1 row in set

Q11. Identify doctors who have not attended any appointments.

Query:

```
SELECT Name FROM Doctors
  WHERE Doctor_ID NOT IN (SELECT DISTINCT Doctor_ID FROM Appointments);
```

Output:

Name
Dr. Priya Nair
Dr. Karan Malhotra
Dr. Sameer Khan
Dr. Pooja Desai
Dr. Manish Agarwal

5 rows in set

Q12. Calculate the cumulative billing revenue per day.

Query:

```
SELECT a.Appointment_Date, SUM(b.Amount) OVER (ORDER BY a.Appointment_Date) AS Cumulative_Revenue
  FROM Appointments a JOIN Billing b ON a.Appointment_ID = b.Appointment_ID;
```

Output:

Appointment_Date	Cumulative_Revenue
2024-01-05	800.00
2024-01-06	1700.00
2024-01-07	2400.00
2024-01-08	2700.00
2024-01-09	3700.00
2024-01-10	4300.00
2024-01-11	4800.00
2024-01-12	5350.00
2024-01-13	6150.00

Appointment_Date	Cumulative_Revenue
2024-01-14	6800.00

10 rows in set

Q13. Find the total revenue generated from each payment method.

Query:

```
SELECT Payment_Method, SUM(Amount) AS Total_Revenue FROM Billing GROUP BY Payment_Method;
```

Output:

Payment_Method	Total_Revenue
Credit Card	3500.00
Cash	1200.00
Debit Card	850.00
UPI	1250.00

4 rows in set

Q14. Display the total amount billed to each patient.

Query:

```
SELECT a.Patient_ID, SUM(b.Amount) AS Total_Billed
  FROM Appointments a JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY a.Patient_ID;
```

Output:

Patient_ID	Total_Billed
1	800.00
3	900.00
5	700.00
7	300.00
10	1000.00
2	600.00
4	500.00
6	550.00
8	800.00
9	650.00

10 rows in set

Q15. Retrieve appointments where more than one medicine was prescribed.

Query:

```
SELECT Appointment_ID FROM Prescriptions
  GROUP BY Appointment_ID HAVING COUNT(Prescription_ID) > 1;
```

Output:

Appointment_ID
3001
3002
3003

3 rows in set

Q16. Show patients who have paid via 'UPI'.

Query:

```
SELECT DISTINCT p.Name FROM Patients p
  JOIN Appointments a ON p.Patient_ID = a.Patient_ID
  JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  WHERE b.Payment_Method = 'UPI';
```

Output:

Name
Simran Kaur
Rajesh Kumar

2 rows in set

Q17. Find the average billing amount per month.

Query:

```
SELECT MONTH(a.Appointment_Date) AS Month, AVG(b.Amount) AS Avg_Amount
  FROM Appointments a JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY Month;
```

Output:

Month	Avg_Amount
1	680.00

1 row in set

Q18. Retrieve the specialization that generated the highest total revenue.

Query:

```
SELECT d.Specialization, SUM(b.Amount) AS Total_Revenue
  FROM Doctors d JOIN Appointments a ON d.Doctor_ID = a.Doctor_ID
  JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY d.Specialization ORDER BY Total_Revenue DESC LIMIT 1;
```

Output:

Specialization	Total_Revenue
Oncology	1000.00

1 row in set

Q19. Retrieve the top 3 patients with the highest total billing amounts.

Query:

```
SELECT a.Patient_ID, SUM(b.Amount) AS Total_Spent
  FROM Appointments a JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY a.Patient_ID ORDER BY Total_Spent DESC LIMIT 3;
```

Output:

Patient_ID	Total_Spent
10	1000.00
3	900.00
1	800.00

3 rows in set

Q20. Find the payment method that has processed the most transactions.

Query:

```
SELECT Payment_Method, COUNT(*) AS Transaction_Count FROM Billing
  GROUP BY Payment_Method ORDER BY Transaction_Count DESC LIMIT 1;
```

Output:

Payment_Method	Transaction_Count
Credit Card	4

1 row in set

Q21. Show the total billing amount generated by each specialization.

Query:

```
SELECT d.Specialization, SUM(b.Amount) AS Total_Revenue
  FROM Doctors d JOIN Appointments a ON d.Doctor_ID = a.Doctor_ID
  JOIN Billing b ON a.Appointment_ID = b.Appointment_ID
  GROUP BY d.Specialization;
```

Output:

Specialization	Total_Revenue
Cardiology	800.00
Neurology	900.00
Orthopedics	700.00
General Medicine	300.00
Oncology	1000.00
Gynecology	600.00
Dermatology	500.00
Ophthalmology	550.00
Gastroenterology	800.00
Psychiatry	650.00

10 rows in set

Q22. List each specialization with its doctor's consultation fee.

Query:

```
SELECT Specialization, MAX(Fee) AS Max_Fee FROM Doctors GROUP BY Specialization;
```

Output:

Specialization	Max_Fee
Cardiology	800.00
Dermatology	500.00
Orthopedics	700.00
Pediatrics	450.00
Neurology	900.00
Gynecology	600.00
ENT	400.00
Ophthalmology	550.00
General Medicine	300.00
Psychiatry	650.00
Urology	750.00
Endocrinology	700.00
Gastroenterology	800.00
Oncology	1000.00
Nephrology	850.00

15 rows in set

Q23. Create an index to search appointments by patient.

Query:

```
CREATE INDEX idx_patient_id ON Appointments(Patient_ID);
DESC Appointments;
```

Output:

Field	Type	Null	Key	Default	Extra
Appointment_ID	int	NO	PRI	NULL	
Patient_ID	int	YES	MUL	NULL	
Doctor_ID	int	YES		NULL	
Appointment_Date	date	YES		NULL	
Status	varchar(20)	YES		NULL	

5 rows in set

Q24. Create a unique index for patient emails.

Query:

```
CREATE UNIQUE INDEX idx_email ON Patients(Email);
DESC Patients;
```

Output:

Field	Type	Null	Key	Default	Extra
Patient_ID	int	NO	PRI	NULL	
Name	varchar(100)	YES		NULL	
Age	int	YES		NULL	
Gender	varchar(10)	YES		NULL	
Contact	varchar(15)	YES		NULL	
Email	varchar(100)	YES	UNI	NULL	
Address	varchar(100)	YES		NULL	

7 rows in set

Q25. Create a view to show bills above $700.

Query:

```
CREATE VIEW HighValueBills AS
  SELECT * FROM Billing WHERE Amount > 700;

SELECT * FROM HighValueBills;
```

Output:

Bill_ID	Appointment_ID	Amount	Payment_Method	Payment_Status
4001	3001	800.00	Credit Card	Paid
4002	3002	900.00	Credit Card	Paid
4005	3005	1000.00	Credit Card	Paid
4009	3009	800.00	Credit Card	Paid

4 rows in set

Q26. Create a view for total revenue per payment method.

Query:

```
CREATE VIEW RevenueSummary AS
  SELECT Payment_Method, SUM(Amount) AS Total_Revenue
  FROM Billing GROUP BY Payment_Method;

SELECT * FROM RevenueSummary;
```

Output:

Payment_Method	Total_Revenue
Credit Card	3500.00
Cash	1200.00
Debit Card	850.00
UPI	1250.00

4 rows in set

SOURCE FILES

- meditrack_schema.sql — table definitions, sample data, and index/view statements
- meditrack_queries.sql — the questions-and-answers query set used in this report

REFERENCES

https://www.w3schools.com/sql/

https://sqlbolt.com/

https://dev.mysql.com/doc/